Bags, Vectors & Transformers

Word Embeddings in Practice · Day 2, Part 2

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Where We Are Headed

We now know *what* embeddings are and *why* they work (the distributional hypothesis). This session is about *how* they are made and *how* you use them — right before you try it yourself.

Part 1 — How they're built

- Word2Vec: learning by prediction
- GloVe: learning from co-occurrence
- Pre-trained vs. training your own

Part 2 — How you use them

- Similarity and nearest neighbors
- From words to documents
- Visualizing the space

Then we break for a hands-on notebook.

Word2Vec: Learning by Prediction

Word2Vec (Mikolov et al., 2013) learns embeddings by training a simple neural network on a **prediction task** — using the distributional hypothesis directly.

The trick: we do not actually care about the predictions. We set up a task that *forces* the model to learn about context, then **throw the task away and keep the weights**. Those weights *are* the embeddings.

The "fake task": slide a window across billions of sentences and, at each position, quiz the model about which words appear near which. To get good at this, the model *must* place words used in similar contexts near each other.

Two Versions: Skip-gram and CBOW

The prediction task comes in two directions:

Skip-gram

Given a word, predict its context.

Input: "**milk**"
Predict: glass, drinks, fridge, sour

- Works well with **rare words**
- Slower, often higher quality

CBOW

Given a context, predict the word.

Input: glass, drinks, fridge, sour
Predict: "**milk**"

- **Faster** to train
- Smooths over frequent words

Same underlying idea, opposite directions. Skip-gram is the more common default.

You Are CBOW

YOUR TURN CBOW sees only the window: two words on each side. Predict the hidden center word from those four words alone.

a cold _____ of milk

she _____ milk every

the angry _____ drove tractors

and the _____ of the

You Are CBOW

YOUR TURN CBOW sees only the window: two words on each side. Predict the hidden center word from those four words alone.

a cold _____ of milk		**glass**
she _____ milk every		
the angry _____ drove tractors		
and the _____ of the		

You Are CBOW

YOUR TURN CBOW sees only the window: two words on each side. Predict the hidden center word from those four words alone.

a cold _____ of milk	**glass**
she _____ milk every	**drinks**
the angry _____ drove tractors	
and the _____ of the	

You Are CBOW

YOUR TURN CBOW sees only the window: two words on each side. Predict the hidden center word from those four words alone.

a cold _____ of milk	**glass**
she _____ milk every	**drinks**
the angry _____ drove tractors	**farmers** (or protesters, men, . . .)
and the _____ of the	

You Are CBOW

YOUR TURN CBOW sees only the window: two words on each side. Predict the hidden center word from those four words alone.

a cold _____ of milk	**glass**
she _____ milk every	**drinks**
the angry _____ drove tractors	**farmers** (or protesters, men, . . .)
and the _____ of the	anything: **rest**, **end**, **future**, . . .

Some windows almost give the word away; others tell you next to nothing. The model can't choose: it learns a little from every window it sees, which is why it needs **so much text**.

How the Window Works (Skip-gram)

A **context window** slides across the text. The center word is the input; the surrounding words are what the model tries to predict.



From the center word **milk**, the model learns to predict neighbors like **drinks** and **every**. Repeat across billions of positions, and words sharing neighbors drift together in the space.

*One detail worth knowing: rather than predict over the whole vocabulary each time (expensive), Word2Vec uses a shortcut called **negative sampling** — distinguishing true neighbors from a few random "fake" ones.*

Make the Training Pairs

YOUR TURN Window size **2** (two words on each side). Which (center, context) pairs does skip-gram create for the center word **tractors**?

angry | farmers | drove | **tractors** | to | parliament

Make the Training Pairs

YOUR TURN Window size **2** (two words on each side). Which (center, context) pairs does skip-gram create for the center word **tractors**?

angry | farmers | drove | **tractors** | to | parliament

(tractors, **farmers**) (tractors, **drove**) (tractors, **to**) (tractors, **parliament**)

Four pairs. "angry" is outside the window, and "to" counts as context too: stop words take part unless you remove them first. Then the window moves one word on, and the next word becomes the center.

GloVe: Learning from Co-occurrence

GloVe (Pennington et al., 2014) reaches a similar goal by a different route. Instead of sliding a prediction window, it looks at **global co-occurrence counts**: how often does each word appear near each other word, across the *whole* corpus?

Word2Vec

- **Predictive**
- Learns from **local** windows
- One position at a time
- Neural-network flavor

GloVe

- **Count-based**
- Learns from **global** statistics
- Factorizes a co-occurrence matrix
- Matrix-factorization flavor

Same goal — similar words, similar vectors. In practice the two give comparable results; the choice is often just which pre-trained set is handy.

Pre-trained vs. Training Your Own

Use pre-trained

· Download vectors trained on **huge** corpora (Wikipedia, news, Google News)

· Free, instant, high quality

· Great default for **general** language

Best when your text is "ordinary".

Train your own

· Fit on **your** corpus

· Captures **domain-specific** meaning

· Useful for **lower-resource languages**

· Needs a reasonably large corpus

Why this matters for social science: in a political or medical corpus, words like "*positive*", "*conservative*", or "*treatment*" can mean something quite different from everyday usage. Domain-trained vectors capture that; generic ones may not.

Predict the Neighbors

YOUR TURN Write down three words you expect among GloVe's nearest neighbors of each word. Then we look.

wageningen

nitrogen

populism

Predict the Neighbors

YOUR TURN Write down three words you expect among GloVe's nearest neighbors of each word. Then we look.

wageningen	nitrogen	populism
giessen	sulfur	populist
groningen	ammonia	conservatism
radboud	chlorine	liberalism
njala	dioxide	demagogue
tilburg	oxides	demagoguery

Wageningen sits among **other university places**. "Nitrogen" is pure **chemistry**: GloVe learned from Wikipedia and news from before 2015, years before the Dutch nitrogen crisis. Pre-trained vectors know the world of their training data, nothing more.

Using Embeddings: Similarity & Neighbors

Once you have vectors, the most basic operation is asking **how similar** two words are. The standard measure is **cosine similarity**: the angle between two vectors.

· Ranges from -1 to 1

· **1** = same direction (very similar)

· **0** = unrelated

· Uses *direction*, not length — so document length does not distort it

Nearest neighbors

The most useful query: "what words are closest to X?"

king → queen, prince, monarch, throne

A quick sanity check that your embeddings make sense.

Cosine similarity + nearest neighbors is how you actually explore an embedding space — you will do a lot of this in the notebook.

From Words to Documents

A practical catch: embeddings are **per word**, but your unit of analysis is usually a **document** (a tweet, a speech, a response). How do you get a vector for a whole document?

Simple & common

· **Average** the word vectors in the document

· Surprisingly effective baseline

· Optionally weight by TF-IDF so rare words count more

More sophisticated

· **Doc2Vec**: learn document vectors directly

· Contextual models (BERT) give document-aware vectors

More on this later.

Averaging loses word order (again!) — but for many tasks it is a strong, simple starting point.

Are These Sentences Similar?

YOUR TURN How similar is **A** to **B**, and **A** to **C**: with bag-of-words, and with averaged GloVe vectors?

A farmers protest new rules

B growers demonstrate against regulations

C museum opens exhibition tomorrow

Are These Sentences Similar?

YOUR TURN How similar is **A** to **B**, and **A** to **C**: with bag-of-words, and with averaged GloVe vectors?

A farmers protest new rules

B growers demonstrate against regulations

C museum opens exhibition tomorrow

cosine similarity	A–B	A–C
bag-of-words	0.00	0.00
averaged GloVe	**0.90**	0.56

A and B share **no words**, so bag-of-words sees nothing in common. Averaged embeddings see the same meaning. But A–C still scores 0.56: averaged vectors are all fairly alike, so **compare** scores rather than reading one on its own.

Seeing the Space: Visualization

Embeddings live in **hundreds of dimensions** — impossible to see directly. To inspect them, we **project down to 2-D** using dimensionality reduction.

Common tools

· **t-SNE** — great for showing clusters

· **UMAP** — faster, preserves more structure

· **PCA** — simple, linear, fast

Handle with care

· 2-D is a **lossy** cartoon of the real space

· Distances can **distort**

· Great for intuition, risky for strong claims

Use visualizations to explore and communicate — not as evidence on their own.

Let’s open the notebook 

Time to load real embeddings, find nearest neighbors, try analogies, and visualize the space yourself.